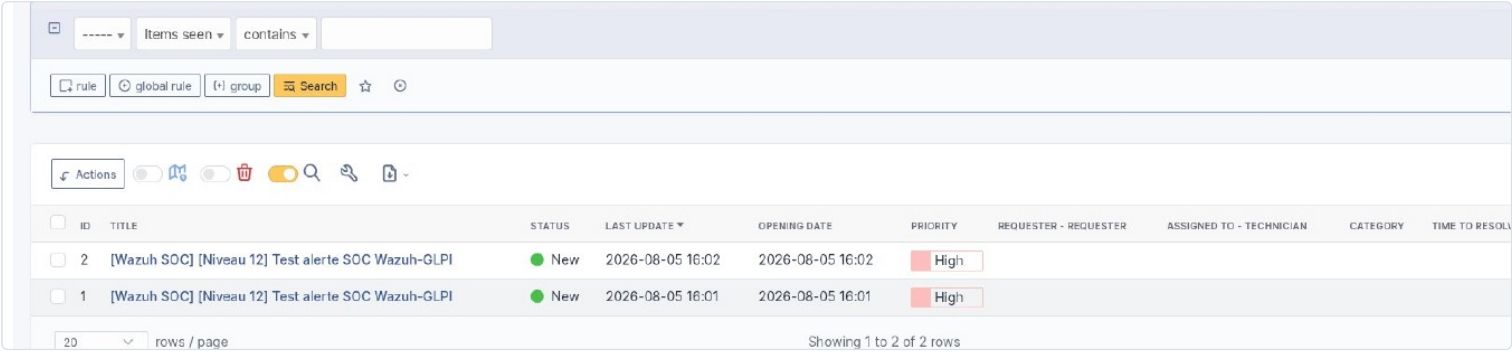


SOC Automation: Integrating Wazuh SIEM with GLPI ITSM

A complete technical guide to setting up automatic incident ticket creation from SIEM alerts.



1. Introduction & Architecture

This document describes the full procedure for interconnecting a Wazuh SIEM manager deployed on a Debian host with a GLPI ITSM instance running inside a Docker container. The goal of this automation is to instantly turn critical security alerts (level 10 and above) into qualified support tickets in GLPI, reducing SOC teams' mean time to respond (MTTR).

2. GLPI Prerequisites & API Tokens

Authenticating against GLPI's REST API requires generating an application token (App-Token) and a user token (User-Token) beforehand. Enable the REST API in GLPI (**Setup > General > API tab**) and create a dedicated API client (e.g. `Wazuh-SOC`). Make sure to correctly configure IP restrictions if needed, to allow the Docker gateway's IP address (e.g. `172.18.0.1`) or the local Wazuh server's IP.

⚠ **Note on secrets:** the tokens below are placeholders. Never commit real App-Token/User-Token values to a public repository — generate your own in GLPI and store them as environment variables or in a secrets manager rather than hardcoding them in the script.

3. Building the Custom Integration Script

Wazuh natively supports script-based integrations executed from the `/var/ossec/integrations/` directory. This custom Python script intercepts the JSON alert payload sent by Wazuh, extracts the relevant fields (title, severity, agent, description), and issues an HTTP POST request to GLPI's REST API to open the ticket.

```
#!/usr/bin/env python3
import sys
import json
import urllib.request
import urllib.error

# Read the alert file provided by Wazuh
alert_file = sys.argv[1]

with open(alert_file, 'r') as f:
    alert_data = json.load(f)

# Extract key alert fields
rule_level = alert_data.get('rule', {}).get('level', 0)
rule_id = alert_data.get('rule', {}).get('id', '')
rule_desc = alert_data.get('rule', {}).get('description', 'Wazuh SOC Alert')
agent_name = alert_data.get('agent', {}).get('name', 'unknown')

# GLPI API connection parameters
glpi_url = "http://127.0.0.1:8080/apirest.php"
app_token = "YOUR_APP_TOKEN"
user_token = "YOUR_USER_TOKEN"

# 1. Initialize the GLPI API session
init_url = f"{glpi_url}/initSession"
req = urllib.request.Request(init_url, headers={
    "Content-Type": "application/json",
    "App-Token": app_token,
    "Authorization": f"user_token {user_token}"
})

try:
    with urllib.request.urlopen(req) as response:
        res_data = json.loads(response.read().decode())
        session_token = res_data.get("session_token")
except Exception as e:
    sys.exit(1)

# 2. Create the support ticket
ticket_url = f"{glpi_url}/Ticket"
ticket_payload = {
    "input": {
        "name": f"[Wazuh SOC] [Level {rule_level}] {rule_desc}",
        "content": f"Security alert detected by Wazuh.\n- Rule ID: {rule_id}\n- Level: {rule_level}\n- Agent: {agent_name}",
        "urgency": 4 if rule_level >= 12 else 3,
        "priority": 4 if rule_level >= 12 else 3
    }
}

req_ticket = urllib.request.Request(ticket_url,
    data=json.dumps(ticket_payload).encode('utf-8'), headers={
        "Content-Type": "application/json",
        "App-Token": app_token,
        "Session-Token": session_token
    })

try:
    with urllib.request.urlopen(req_ticket) as response:
        pass
except Exception as e:
    sys.exit(1)

# 3. Cleanly close the GLPI session
kill_url = f"{glpi_url}/killSession"
req_kill = urllib.request.Request(kill_url, headers={
    "App-Token": app_token,
    "Session-Token": session_token
})

try:
    with urllib.request.urlopen(req_kill):
        pass
except Exception:
    pass
```

4. Deployment & Permissions

Save the script as `custom-glpi.py` in Wazuh's integrations directory, then apply strict ownership/permissions so it can only be run by the Wazuh daemon:

```
chmod 755 /var/ossec/integrations/custom-glpi.py
chown root:wazuh /var/ossec/integrations/custom-glpi.py
```

5. Wazuh Manager Configuration (`ossec.conf`)

Edit Wazuh Manager's main configuration file (`/var/ossec/etc/ossec.conf`) to enable the integration hook for alerts at or above level 10:

```
<integration>
  <name>custom-glpi</name>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

Restart the Wazuh Manager service to apply the new configuration:

```
systemctl restart wazuh-manager
```

6. Testing & Validating the Chain

To validate the integration without waiting for a real incident, simulate a level-12 alert by injecting a test JSON file and manually running the script:

```
echo '{"rule":{"level":12,"id":"100001","description":"Wazuh-GLPI SOC test alert"},"agent":{"name":"Debian-Test"}}' >
/tmp/test_alert.json
python3 /var/ossec/integrations/custom-glpi.py /tmp/test_alert.json
```

Then log in to the GLPI interface, in **Assistance > Tickets**, to confirm the ticket was created automatically with the correct severity and impacted-agent information.